DuckDB extensions

The Query.Farm quick reference collection.

32 extension guides. One consistent reference.

Install commands, core APIs, practical SQL and usage notes. Page numbers refer to the combined volume. Each title also links to its standalone PDF.

A5	2	JSONata	18
ADBC Scanner	3	Lindel	19
Airport	4	Marisa	20
Bitfilters	5	MiniJinja	21
Cronjob	6	QuickJS	22
Crypto	7	Radio	23
Apache DataSketches	8	RapidFuzz	24
Rhai	9	Redis	25
Events	10	Shellfs	26
Fuzzycomplete	11	Stochastic	27
GeoSilo	12	Tera	29
Hashfuncs	13	Textplot	30
HTTP Client	14	Tributary	31
HTTP Server	15	TSID	32
Inflector	16	VGI	33
JSON Schema	17	WebMacro	34

Using this collection

Examples marked “Requires setup” need the service, driver or credentials named in the guide. “Manual setup” marks CLI, driver or process-lifecycle steps. Local examples were executed against the builds recorded in each footer; source-reviewed guides identify compatibility limits.

Browse the complete extension catalog ↗

A5

Equal-area cells for geographic data.

Install & load

```
INSTALL a5 FROM community;
LOAD a5;
```

01 At a glance

Turn longitude and latitude into pentagonal cell IDs for grouping, joining, and geographic rollups. A5 uses equal-area cells at each resolution, with integer IDs and functions for hierarchy, neighborhoods, and geometry conversion.

Longitude / latitude → A5 cell ID → Group / join / roll up

02 Core SQL surface

`a5_lonlat_to_cell(lon,lat,res)`

Map longitude, latitude in degrees to a UBIGINT cell ID.

`a5_cell_to_lonlat(cell)`

Return the cell center as [longitude, latitude], not the original point.

`a5_cell_to_parent(cell,res)`

Find a coarser ancestor for hierarchical rollups.

`a5_cell_area(res)`

Return equal cell area in square meters at a resolution.

`a5_grid_disk(cell,k)`

Return cells within k edge steps, including the center cell.

`a5_cell_to_geometry(cell)`

Produce a native GEOMETRY polygon for mapping.

`a5_geometry_to_cells(geom,res)`

Cover geometry; polygons return a compacted covering.

03 Keep in mind

Coordinates and identifiers

Pass longitude first, latitude second, in degrees. Resolutions run from 0 to 30; larger values mean smaller cells. Preserve IDs as UBIGINT or use a5_u64_to_hex when exporting to systems with limited integer precision.

04 Encode New York City

```
SELECT a5_lonlat_to_cell(
  -74.0060, 40.7128, 15
) AS cell;
SELECT a5_cell_area(15) AS area_m2;
```

Cell 2760559170984869888; area ≈31,669 m².

05 Count observations by cell

```
WITH points(lon,lat) AS (VALUES
  (-74.0060,40.7128), (-74.0061,40.7129),
  (-73.9857,40.7484)
)
SELECT a5_lonlat_to_cell(lon,lat,15) AS cell,
       count(*) AS n
FROM points
GROUP BY cell;
```

Equal-resolution cell IDs are ordinary grouping and join keys.

06 Roll up and draw a cell

```
WITH p AS (
  SELECT a5_lonlat_to_cell(
    -74.0060,40.7128,15) AS cell
)
SELECT a5_cell_to_parent(cell,10) AS parent,
       a5_cell_to_geometry(cell) AS polygon
FROM p;
```

The parent is a hierarchy key; the polygon is native GEOMETRY.

07 In practice

Render at one resolution

Coverings from a5_geometry_to_cells, a5_grid_disk, and a5_compact end in a compaction marker that is not a cell. Expand with a5_uncompact(cells) before unnesting or drawing them. A5 1.0 changed cell IDs: convert stored pre-1.0 IDs once with a5_migrate.

Full docs & examples ↗ Source ↗

ADBC Scanner

Database drivers. Arrow batches. Ordinary SQL.

Install & load

```
INSTALL adbc_scanner FROM community;
LOAD adbc_scanner;
```

01 At a glance

Connect DuckDB to a database through an Arrow Database Connectivity driver. Use attached catalogs for joins and discovery, send backend SQL explicitly, or bulk-ingest a DuckDB query through the driver's Arrow interface.

ADBC driver → Arrow batches → DuckDB SQL

02 ATTACH API essentials

`ATTACH … (TYPE adbc)`

Open a catalog; pass its alias to the functions below.

`adbc_scan(db, sql)`

Execute backend SQL; supply params and columns when needed.

`adbc_scan_table(db, table)`

Read a table with schema discovery and supported pushdown.

`CALL adbc_execute(db, sql)`

Run backend DDL or DML; return affected-row count.

`adbc_insert(db, table, query)`

Ingest Arrow batches; modes include create, append and replace.

`adbc_schema(db, table)`

Describe a named table, rather than an arbitrary query.

`CALL adbc_clear_cache()`

Refresh attached catalog metadata after remote DDL changes.

03 Keep in mind

Match the API revision

These examples target the v1.5 ATTACH API introduced in source revision 7e5210c. Older community builds use BIGINT connection handles instead. Install a matching build plus an ADBC driver manifest or shared library.

04 Query through a driver

MANUAL SETUP

```
ATTACH ':memory:' AS db (
  TYPE adbc, driver 'sqlite'
);
SELECT * FROM adbc_scan(
  'db', 'SELECT ? AS id',
  params := row(42),
  columns := {'id': 'BIGINT'}
);
```

Requires the ATTACH API build and SQLite ADBC driver. Returns id = 42.

05 Bulk load a DuckDB result

MANUAL SETUP

```
SELECT * FROM adbc_insert(
  'db', 'people',
  (SELECT 1::BIGINT AS id, 'Ada' AS name),
  mode := 'create'
);
SELECT * FROM adbc_scan_table('db', 'people');
SELECT * FROM adbc_schema('db', 'people');
DETACH db;
```

Continues the first example; ingestion modes depend on the driver.

06 In practice

Schemas and transactions

SQLite needs explicit columns for arbitrary scans. BEGIN / COMMIT controls the attachment's remote transaction; it does not create a distributed transaction. Use a file for SQLite catalog scans and transactions: separate :memory: connections are separate databases.

Full docs & examples ↗ Source ↗

Airport

Bring Arrow Flight services into your SQL.

Install & load

```
INSTALL airport FROM community;
LOAD airport;
```

01 At a glance

Attach an Arrow Flight service as a DuckDB catalog, then query its tables alongside local data. Discover available flights, fetch a descriptor directly, and invoke server-defined actions when a full catalog is unnecessary.

Flight server → Arrow stream → DuckDB catalog

02 Catalogs, flights & actions

`ATTACH … (TYPE airport)`

Mount a compatible Flight catalog under a DuckDB alias.

`airport_databases(url)`

Discover databases exposed by the server.

`airport_flights(url)`

List available flight descriptors and metadata.

`airport_take_flight(url, descriptor)`

Read a specific flight directly as a table.

`airport_action(url, action, body)`

Invoke a server-defined action; available actions vary.

`CREATE SECRET … (TYPE airport)`

Scope a bearer token to a Flight URL prefix.

`airport_version()`

Identify the loaded Airport build for diagnostics.

03 Keep in mind

Capabilities come from the server

Catalog discovery, filter pushdown and writes depend on the Flight server's Airport-compatible interfaces. A service using Arrow internally does not necessarily expose those interfaces. Check its schema and capabilities before attaching.

04 Discover before attaching

REQUIRES SETUP

```
SELECT * FROM airport_databases(
  'grpc://localhost:8815'
);
SELECT * FROM airport_flights(
  'grpc://localhost:8815'
);
```

Requires a local compatible Flight server listening on port 8815.

05 Query an attached catalog

REQUIRES SETUP

```
ATTACH 'grpc://localhost:8815' AS remote
  (TYPE airport);
SHOW TABLES FROM remote.public;
SELECT user_id, count(*) AS n
FROM remote.public.activity
GROUP BY user_id
ORDER BY n DESC
LIMIT 10;
DETACH remote;
```

Assumes the server exposes public.activity with a user_id column.

06 Identify the loaded build

```
SELECT airport_version(), airport_user_agent();
```

Local diagnostics; no server connection required.

07 In practice

Authenticate by URL scope

Use an airport secret with scope and auth_token for protected services. Use grpc+tls for TLS endpoints. Server-defined descriptors, table names and actions must match the service you connect to.

Full docs & examples ↗ Source ↗

Bitfilters

Compact membership checks for large sets.

Install & load

```
INSTALL bitfilters FROM community;
LOAD bitfilters;
```

01 At a glance

Aggregate hashed keys into a compact BLOB, then probe it to reject definite nonmembers. Use an exact check for candidates that survive the filter.

Hash keys → Build filter → Probe & verify

02 Core SQL surface

`xor8_filter(hash)`

Aggregate UBIGINT hashes into a static XOR filter.

`xor8_filter_contains(filter, hash)`

Return whether a key might be in the set.

`xor16_filter / xor16_filter_contains`

Use wider fingerprints for fewer false positives.

`binary_fuse8_filter / binary_fuse16_filter`

Alternative static filters with matching _contains probes.

`quotient_filter(q, r, hash)`

Build a quotient filter; q and r control capacity and precision.

`quotient_filter_contains(filter, hash)`

Probe a quotient filter with the same hash representation.

03 Keep in mind

A positive is only a candidate

Keep an exact join or membership predicate after probing. False positives are expected; correctly built filters retain inserted keys.

04 Build once

```
CREATE TABLE known_filter AS
SELECT xor8_filter(hash(i)) AS f
FROM range(1000) AS t(i);
```

Constructors aggregate rows; the result is a BLOB you can persist.

05 Probe candidates

```
SELECT i, xor8_filter_contains(f, hash(i))
  AS maybe_known
FROM known_filter, range(995, 1005) AS t(i);
```

Inserted keys return true. An unseen key can also return true.

06 Confirm the match

```
SELECT i
FROM known_filter, range(995, 1005) AS t(i)
WHERE xor8_filter_contains(f, hash(i))
  AND i IN (SELECT range FROM range(1000));
```

The final exact predicate returns only IDs 995 through 999.

07 In practice

Hash consistently

Use the same function, seed and SQL input type when building and probing. Treat DuckDB hash() as version-dependent; persist its version or choose an explicit hash algorithm.

Full docs & examples ↗ Source ↗

Cronjob

Schedule recurring SQL in a live DuckDB process.

Install & load

```
INSTALL cronjob FROM community;
LOAD cronjob;
```

01 At a glance

Register SQL with a six-field cron expression, inspect jobs and cancel by ID. The scheduler lives inside the DuckDB process, making it useful for short-lived local automation.

SQL + schedule → Live scheduler → Inspect / cancel

02 Core SQL surface

`cron(sql, schedule)`

Register a query; returns the job ID as VARCHAR.

`cron_jobs()`

Table function listing scheduled jobs and their status.

`cron_delete(job_id)`

Cancel a job using the ID returned by cron.

`second minute hour day month weekday`

Six fields: seconds come first, unlike five-field system cron.

`0 */5 * * * *`

Every five minutes, at second zero.

`0 0 2 * * *`

Daily at 02:00:00 in the scheduler environment.

03 Keep in mind

Keep the process alive

The scheduler stops when DuckDB exits. Register jobs again after a restart; this is not a persistent system service or a replacement for an external orchestrator.

04 Register a job

```
CREATE TEMP TABLE scheduled AS
SELECT cron(
  'SELECT 42', '0 */5 * * * *'
) AS job_id;
```

The example retains the actual ID so cancellation remains reliable.

05 Inspect jobs

```
SELECT * FROM cron_jobs();
```

Review schedule and status while the same DuckDB process remains running.

06 Cancel the registered job

```
SELECT cron_delete(job_id) AS removed
FROM scheduled;
```

Cancel the example job when finished; no future execution remains scheduled.

07 In practice

Keep and use the returned ID

Do not assume a job will always be task_0. Store the returned identifier and use it for cancellation. A successful registration does not establish that the scheduled SQL will succeed.

Full docs & examples ↗ Source ↗

Crypto

Digests, message authentication & random bytes.

Install & load

```
INSTALL crypto FROM community;
LOAD crypto;
```

01 At a glance

Compute cryptographic digests and HMACs directly from SQL values. Use secure random bytes for tokens and salts, or aggregate values in a specified order for reproducible checksums.

SQL values → Hash / HMAC → BLOB or hex

02 Core SQL surface

`crypto_hash(algorithm, value)`

Return a digest as BLOB; algorithm is a string.

`crypto_hmac(algorithm, key, message)`

Authenticate a message with a secret key; returns BLOB.

`crypto_random_bytes(length)`

Generate the requested number of cryptographically secure random bytes.

`crypto_hash_agg(algorithm, value ORDER BY ...)`

Aggregate values into a digest with explicit input order.

`to_hex(blob) / lower(text)`

Format binary digests as printable lowercase hexadecimal.

03 Keep in mind

Choose the algorithm explicitly

Common names include blake3, blake2b-512, sha2-256, sha2-512 and sha3-256. MD5 and SHA-1 are legacy choices, unsuitable for collision-resistant security.

04 Hash a string

```
SELECT lower(to_hex(
  crypto_hash('sha2-256', 'hello')
)) AS digest;
```

SHA-256 returns 32 bytes, rendered here as 64 hex characters.

05 Authenticate a message

```
SELECT lower(to_hex(crypto_hmac(
  'sha2-256', 'example-key', 'message'
))) AS signature;
```

Argument order is algorithm, key, message. Replace the example key.

06 Bytes & ordered aggregates

```
SELECT octet_length(crypto_random_bytes(32));

SELECT to_hex(crypto_hash_agg(
  'sha2-256', s ORDER BY s
)) AS digest
FROM (VALUES ('A'), ('B')) AS t(s);
```

ORDER BY belongs inside the aggregate call for reproducible ordering.

07 In practice

A checksum needs a data contract

Specify row order, input types and serialization. A digest of one field is not automatically an unambiguous digest of a whole row. Fast hashes are a separate hashfuncs extension.

Full docs & examples ↗ Source ↗

Apache DataSketches

Approximate answers from compact aggregate state.

Install & load

```
INSTALL datasketches FROM community;
LOAD datasketches;
```

01 At a glance

Build sketches over rows, persist their typed state, and query estimates later. Select a family for distinct counts, quantiles or frequent items; merge states to summarize partitions.

Rows → Sketch state → Merge / estimate

02 Core SQL surface

`datasketch_hll / datasketch_cpc`

Approximate distinct-count sketches; query with _estimate.

`datasketch_theta`

Distinct-count sketch with union, intersection and difference.

`datasketch_kll`

Quantile sketch; query with _quantile, _rank, _cdf or _pmf.

`datasketch_quantiles / datasketch_req`

Alternative quantile families with different error behavior.

`datasketch_tdigest`

Compact distribution summary for quantile estimates.

`datasketch_frequent_items`

Track frequent strings or numbers and their estimated counts.

`datasketch_hll_union(lg_k, sketch)`

Aggregate compatible HLL states before estimating the union.

03 Keep in mind

Merge states, not estimates

Adding daily distinct estimates double-counts people present on multiple days. Merge compatible sketches first, then ask for the final estimate.

04 Count distinct values

```
SELECT datasketch_hll_estimate(
  datasketch_hll(12, i % 1000)
) AS distinct_values
FROM range(10000) AS t(i);
```

An approximate count of 1,000 distinct values from 10,000 input rows.

05 Estimate p50 and p95

```
WITH agg AS (
  SELECT datasketch_kll(200, i::DOUBLE) AS s
  FROM range(10000) AS t(i)
)
SELECT datasketch_kll_quantile(s, 0.50, true)
         AS p50,
       datasketch_kll_quantile(s, 0.95, true)
         AS p95
FROM agg;
```

The trailing true requests inclusive search. Input type determines sketch type.

06 Inspect retained state

```
SELECT datasketch_kll_n(
  datasketch_kll(200, i::DOUBLE)
) AS input_count
FROM range(10000) AS t(i);
```

The sketch tracks the number of input items, independently of its retained sample.

07 In practice

Match accuracy to the workload

lg_k controls HLL size; k controls KLL capacity. Larger settings use more memory. Quantile rank error is not an absolute error bound on the returned value.

Full docs & examples ↗ Source ↗

Rhai

Evaluate expressions and rules stored as text

Install & load

```
INSTALL evalexpr_rhai FROM community;
LOAD evalexpr_rhai;
```

01 At a glance

Run Rhai expressions inside a query, pass row values as JSON context, and read a typed success or error result. Useful for configurable business rules, small transformations, and scripts that need local variables or branching.

Script text → Row context → JSON result

02 SQL entry point + Rhai syntax

`evalexpr_rhai(script)`

Evaluate a script without row context.

`evalexpr_rhai(script, context)`

Expose a JSON object as context inside the script.

`.ok`

JSON result on success; NULL when the script fails.

`.error`

Error text on failure; NULL when the script succeeds.

`context.field`

Read a field supplied by the SQL query.

`let / if / fn`

Bind variables, branch, and define helpers inside scripts.

03 Keep in mind

> **Keep the boundary explicit**
>
> The result is UNION(ok JSON, error VARCHAR). Cast .ok to a SQL type before arithmetic or joins. Inspect .error to distinguish a failed script from a successful JSON null.

04 Evaluate and cast

```
SELECT evalexpr_rhai(
  'fn sq(x) { x * x } sq(context.n)',
  {'n': 7}
).ok::INTEGER AS square;
-- 49
```

Define helpers in the script; return its final expression.

05 Apply one rule to each row

```
SELECT salary, evalexpr_rhai(
  'context.salary ⩾ 100000',
  {'salary': salary}
).ok::BOOLEAN AS eligible
FROM (VALUES (80000), (120000)) t(salary);
-- false, true
```

Use a constant rule with a different JSON context for each row.

06 Read a failure without losing the query

```
SELECT evalexpr_rhai('1 / 0').error;
-- Runtime error text
```

A failing Rhai expression returns the error arm.

07 In practice

> **Use it where rules change**
>
> Scripts run per row. Keep routine transformations in native SQL; use Rhai when stored rules or branching improve clarity. A constant script can reuse its parsed expression while context changes.

Full docs & examples ↗ Source ↗

Events

Query and transaction events as a JSON stream.

Install & load

```
INSTALL events FROM community;
LOAD events;
```

01 At a glance

Forward DuckDB lifecycle events to an executable handler as JSON lines on standard input. Use the stream for operational logs, query timing and transaction traces, with a session tag to identify each running process.

DuckDB event → JSON line → Handler stdin

02 Four settings & event keys

`events_destination`

Absolute path to the executable that consumes JSON lines.

`events_types`

List of enabled query, connection or transaction event types.

`events_session_name`

Tag each event with a process or workload identifier.

`events_async`

TRUE enables asynchronous delivery; default is FALSE.

`query_id / transaction_id`

Correlate query pairs and the enclosing transaction.

`has_error / error_message`

Inspect failures on query_end and transaction_rollback events.

03 Keep in mind

> **A handler is required**
>
> Create an executable with a shebang at the configured absolute path. It must read stdin line by line and persist or process each JSON object. The extension launches the handler with DuckDB's privileges.

04 Capture query activity

MANUAL SETUP

```
SET events_session_name = 'analytics';
SET events_types = [
  'query_begin', 'query_end'
];
SET events_async = TRUE;
SET events_destination =
  '/usr/local/bin/query-events';
SELECT sum(i) FROM range(1000) t(i);
```

Requires an installed compatible Events build and your executable JSONL handler.

05 Trace transaction boundaries

MANUAL SETUP

```
SET events_types = [
  'transaction_begin',
  'transaction_commit',
  'transaction_rollback'
];
BEGIN;
SELECT 42;
COMMIT;
```

With the handler running, group transaction events by transaction_id.

06 In practice

> **Lifecycle and availability**
>
> Settings belong to the running process; reapply them at startup. Async delivery trades reliability for query latency. Persist events in the handler when durability matters. Community package availability varies by DuckDB version and platform.

Full docs & examples ↗ Source ↗

Fuzzycomplete

Fuzzy SQL completion with a catalog-aware matcher

Aa

Install & load

```
INSTALL fuzzycomplete FROM community;
LOAD fuzzycomplete;
```

01 At a glance

Suggest SQL keywords and catalog names from a partial query using a fuzzy matcher. Fixed v1.5 builds activate even when the DuckDB CLI has already loaded stock autocomplete; inspect suggestions through the table function.

Partial SQL → Catalog + matcher → Suggestions

02 Completion surface

`sql_auto_complete(query)`

Table function: suggest completions for partial SQL text.

`suggestion`

Candidate text returned by the completion provider.

`suggestion_start`

Character offset at which the candidate should replace text.

`Tab in the CLI`

Requests completion through the active provider.

`duckdb_extensions()`

Check whether autocomplete was already loaded.

`duckdb_functions()`

Inspect the registered completion function and its parameters.

03 Keep in mind

> **Activation depends on the build**
>
> Fixed v1.5 builds replace preloaded stock autocomplete. Older 9746833 skips replacement when autocomplete is loaded. Check behavior with SLCT: the fuzzy matcher suggests SELECT. Loaded=true alone does not identify the active provider.

04 Inspect the loaded provider

```
SELECT extension_name, loaded
FROM duckdb_extensions()
WHERE extension_name IN
  ('autocomplete', 'fuzzycomplete');
```

Reports loaded extensions; use the suggestion check below to verify fuzzy behavior.

05 Inspect SQL suggestions

```
SELECT trim(suggestion) AS candidate
FROM sql_auto_complete('SLCT');
-- SELECT
```

A subsequence query checks the actual fuzzy provider on the fixed build.

06 Try a catalog abbreviation

MANUAL SETUP

```
CREATE TEMP TABLE getNotificationStatus(
  id INTEGER
);
-- In a compatible interactive CLI, type:
-- SELECT * FROM gnst
-- Then press Tab before submitting.
```

The client must call DuckDB's active completion provider for Tab.

07 In practice

> **Matcher and client are separate**
>
> Fuzzy matching rewards ordered character matches and word boundaries. SQL calls inspect suggestions; they do not add an editor to Python, JDBC, or other clients. Interactive Tab behavior depends on the client and active provider.

Full docs & examples ↗ Source ↗

GeoSilo

Compact geometry with explicit precision.

Install & load

```
INSTALL geosilo FROM community;
LOAD geosilo;
```

01 At a glance

Store geometries as scaled integer coordinates with delta encoding. GeoSilo provides a GEOSILO column type, direct measurements and bounds, and explicit encode/decode functions for BLOB pipelines. Choose precision before comparing storage size or query performance.

Native GEOMETRY → Scale + delta encode → Store / inspect / decode

02 Core SQL surface

`GEOSILO('EPSG:3857')`

Typed geometry storage with CRS metadata and automatic coordinate scale.

`geosilo_encode(geom[,scale])`

Encode GEOMETRY to BLOB with automatic or explicit integer scale.

`geosilo_decode(blob)`

Decode BLOB or GEOSILO to a native GEOMETRY value.

`geosilo_metadata(blob)`

Inspect geometry_type, vertex_type, and scale without decoding the geometry.

`ST_Area / ST_Perimeter`

Measure GEOSILO polygons directly; results use the coordinate system's units.

`ST_XMin / ST_XMax`

Read X bounds; ST_YMin and ST_YMax provide the Y bounds.

`ST_GeometryType / ST_NPoints`

Inspect the geometry kind and count of vertices.

03 Keep in mind

Precision follows the scale

Encoding rounds coordinates onto a grid with spacing 1/scale. A scale of 100 keeps hundredths of a coordinate unit. Projected-meter defaults use 100; degrees default to 10^7. CRS labeling does not transform coordinates.

04 Store a projected polygon

```
CREATE TABLE plots (
  geom GEOSILO('EPSG:3857')
    USING COMPRESSION zstd
);
INSERT INTO plots VALUES (
  'POLYGON((0 0,10 0,10 10,0 10,0 0))'
    ::GEOMETRY
);
```

Inserting GEOMETRY automatically encodes it for the typed column.

05 Inspect and measure directly

```
SELECT ST_Area(geom) AS area,
       ST_Perimeter(geom) AS perimeter,
       geosilo_metadata(geom) AS metadata
FROM plots;
```

Area 100; perimeter 40; integer scale 100. These are planar measurements.

06 Set a scale for a BLOB pipeline

```
WITH encoded AS (
  SELECT geosilo_encode(
    'POINT(1.234 2.345)'::GEOMETRY,100
  ) AS blob
)
SELECT geosilo_decode(blob)::VARCHAR AS wkt
FROM encoded;
```

Returns POINT (1.23 2.35): decoding retains the chosen quantization.

07 In practice

Spatial compatibility

Fixed v1.5 builds merge shared ST_* overloads, so either extension can load first. These examples pass with spatial loaded first on DuckDB 1.5.6. Older GeoSilo d088741 collides with spatial 04270fe; upgrade or use geosilo alone.

Full docs & examples ↗ Source ↗

Hashfuncs

Fast fingerprints, buckets & cache keys.

Install & load

```
INSTALL hashfuncs FROM community;
LOAD hashfuncs;
```

01 At a glance

Hash SQL values with named non-cryptographic algorithms. Choose a stable representation and seed for repeatable partitioning, probabilistic-filter inputs and compact fingerprints.

Canonical input → Named hash → Bucket / key

02 Core SQL surface

`xxh3_64(value [, seed])`

Fast 64-bit result as UBIGINT.

`xxh3_128(value [, seed])`

128-bit result as UHUGEINT.

`xxh3_128_hex(text [, seed])`

Canonical 32-character lowercase hex digest for strings and BLOBs.

`xxh32 / xxh64`

Classic xxHash variants, with optional seed overloads.

`rapidhash(value [, seed])`

64-bit RapidHash for general-purpose fingerprints.

`murmurhash3_32 / murmurhash3_x64_128`

Named MurmurHash3 variants for explicit algorithm selection.

03 Keep in mind

Keep the representation fixed

Match input bytes, type and seed. Fixed v1.5 hex output uses canonical high64 then low64. Older 0dec806 reverses those halves. Numeric UHUGEINT output retains its legacy layout for stored-key compatibility; use the hex function for interchange.

04 Route to 16 buckets

```
SELECT i, xxh3_64(i, 42) % 16 AS bucket
FROM range(8) AS t(i);
```

The same input type and seed assign each value to the same bucket.

05 Format a string fingerprint

```
SELECT xxh3_128_hex('hello') AS digest;
-- 32 lowercase hexadecimal characters
```

Fixed v1.5 builds match canonical xxHash digests directly.

06 Choose the output width

```
SELECT typeof(xxh3_64('hello')) AS small,
       typeof(xxh3_128('hello')) AS large;
-- UBIGINT, UHUGEINT
```

Choose the width for your collision budget and storage format.

07 In practice

Collisions are possible

Hashes are compact fingerprints, not uniqueness constraints. Retain original keys for exact equality. Use crypto for authentication or adversarial inputs.

Full docs & examples ↗ Source ↗

HTTP Client

Small HTTP requests, directly from DuckDB.

Install & load

```
INSTALL http_client FROM community;
LOAD http_client;
```

01 At a glance

Fetch HTTP resources or submit JSON and form bodies from a SQL expression. Responses arrive as a JSON envelope containing status, reason and body, ready to inspect or save in a local table.

HTTP request → JSON envelope → SQL projection

02 Requests & response fields

`http_get(url)`

Fetch a response; a three-argument overload accepts maps.

`http_head(url)`

Request response metadata without downloading the body.

`http_post(url, headers, json)`

Send a JSON body; headers are MAP(VARCHAR, VARCHAR).

`http_post_form(url, headers, params)`

Send form fields supplied as a VARCHAR-to-VARCHAR map.

`response->>'status'`

Read the HTTP status; cast to INTEGER for comparisons.

`response->>'body'`

Extract the body string; cast to JSON when appropriate.

03 Keep in mind

Materialize responses

Requests are synchronous and buffer the response. Save responses to a table before repeated analysis. Deduplicate input keys for enrichment: a function evaluation can trigger a network call, and SQL evaluation order is not a request scheduler.

04 Fetch once, inspect locally

REQUIRES SETUP

```
CREATE TEMP TABLE response AS
SELECT http_get('https://example.com/') AS r;
SELECT (r->>'status')::INTEGER AS status,
       r->>'reason' AS reason,
       r->>'body' AS body
FROM response;
```

Requires network access. The example.com response is HTML, so keep body as text.

05 POST a JSON document

REQUIRES SETUP

```
SELECT http_post(
  'https://api.example.com/events',
  MAP {'content-type': 'application/json'},
  '{"event":"ready","rows":42}'::JSON
);
```

Replace the URL with your POST endpoint and add required authentication headers.

06 Decode a JSON response body

```
WITH r AS (
  SELECT '{"status":200,
    "body":"{\"count\":42}"}'::JSON AS v
)
SELECT ((v->>'body')::JSON)->>'count' AS n
FROM r;
```

Offline envelope fixture; returns the text value 42.

07 In practice

Inspect status before parsing

A body can be HTML, plain text or JSON. Check status and expected content before casting it. Endpoint authentication, retries and rate limits belong in the surrounding workflow; do not assume an HTTP error is a successful API result.

Full docs & examples ↗ Source ↗

HTTP Server

Serve a DuckDB connection through HTTP.

Install & load

```
INSTALL httpserver FROM community;
LOAD httpserver;
```

01 At a glance

Start an HTTP listener inside a DuckDB process and query it from an HTTP client. Send SQL in a POST body, receive JSON results, and keep the process running for the lifetime of the service.

HTTP client → SQL request → DuckDB result

02 Server calls & HTTP contract

`httpserve_start(host, port, auth)`

Start the in-process listener with one shared credential.

`httpserve_stop()`

Stop the listener; call start again to change settings.

`POST /`

Put SQL in the request body to execute it.

`GET /?q=…`

Pass URL-encoded SQL as a query parameter.

`GET /ping`

Unauthenticated liveness endpoint.

`default_format=JSONEachRow`

Return newline-delimited JSON objects, one per result row.

`default_format=JSONCompact`

Return metadata, row arrays and query statistics.

03 Keep in mind

Choose the authentication mode

An auth value containing user:pass selects HTTP Basic; a shared key uses X-API-Key. An empty string disables authentication. The listener speaks HTTP: use a TLS proxy for access beyond a trusted local environment.

04 Start a local listener

MANUAL SETUP

```
SELECT httpserve_start(
  'localhost', 9999, 'replace-this-key'
);
```

Run in a dedicated DuckDB process and leave it open. Replace the example key.

05 Read from another DuckDB

REQUIRES SETUP

```
INSTALL httpfs;
LOAD httpfs;
CREATE SECRET quickref_http (
  TYPE HTTP,
  SCOPE 'http://localhost:9999/',
  EXTRA_HTTP_HEADERS MAP {
    'X-API-Key': 'replace-this-key'
  }
);
SELECT * FROM read_json_auto(
  'http://localhost:9999/?q=SELECT%2042'
  || '&default_format=JSONEachRow'
);
```

Use a second process after the listener starts; the matching key is required.

06 Stop from the serving process

MANUAL SETUP

```
SELECT httpserve_stop();
```

The listener also stops when its DuckDB process exits.

07 In practice

Scope the DuckDB process

Clients can submit SQL using the served connection's capabilities. Bind to localhost for development; configure DuckDB permissions and filesystem access for deployment. Read-only database access alone is not a complete SQL sandbox. Keep the process alive.

Full docs & examples ↗ Source ↗

Inflector

Normalize names, keys, and query result columns

Install & load

```
INSTALL inflector FROM community;
LOAD inflector;
```

01 At a glance

Apply consistent naming conventions to strings, struct keys, or every column of a query. Combine case conversion with English inflection helpers when cleaning imported schemas, generating identifiers, or shaping data for another system.

Mixed names → Choose a format → Consistent shape

02 Core SQL surface

`inflect(format, value)`

Transform a string or rename keys in a struct.

`FROM inflect(format, (query))`

Rename all result columns from a query.

`inflector_to_snake_case(text)`

Convert a single name to snake_case.

`inflector_to_camel_case(text)`

Convert a single name to camelCase.

`inflector_to_table_case(text)`

Build a plural snake_case table name.

`inflector_to_plural(word)`

Apply English pluralization rules.

`inflector_to_singular(word)`

Apply English singularization rules; check irregular edge cases.

03 Keep in mind

Select the shape you mean

inflect changes string contents, struct field names, or result column names according to the input type. Values inside a renamed struct or query remain unchanged. Common formats include snake, camel, pascal, and kebab.

04 Normalize nested data keys

```
SELECT inflect('snake', {
  'firstName': 'Ada',
  'lastName': 'Lovelace'
});
-- {first_name: Ada, last_name: Lovelace}
```

Pass a STRUCT; field names change while their values stay intact.

05 Rename a complete result

```
SELECT * FROM inflect('snake', (
  SELECT 'Ada' AS firstName,
         'Lovelace' AS lastName
));
-- columns: first_name, last_name
```

Replace the inner SELECT with your import or transformation query.

06 Derive table and word names

```
SELECT inflector_to_table_case('UserAccount');
-- user_accounts
SELECT inflector_to_plural('person');
-- people
SELECT inflector_to_singular('people');
-- person
```

Use scalar helpers for individual naming decisions.

07 In practice

Check English edge cases

Inflection is English-specific and is not always reversible. Fixed v1.5 builds singularize people to person and children to child; older ac9b69f leaves these unchanged. Validate the vocabulary and acronym conventions that matter to your data.

Full docs & examples ↗ Source ↗

JSON Schema

Validate JSON contracts and apply declared defaults

Install & load

```
INSTALL json_schema FROM community;
LOAD json_schema;
```

01 At a glance

Check JSON documents against a schema inside SQL, then fill missing properties from schema defaults. Useful at ingest and during audits. Invalid documents raise errors in this build, so choose strict validation or a deliberate TRY wrapper.

Schema + JSON → Validate → Patch or enrich

02 Core SQL surface

`json_schema_validate(schema, data)`

Return TRUE on success; raise an error on invalid data.

`json_schema_validate_schema(schema)`

Check schema acceptance; invalid schemas can raise errors.

`json_schema_patch(schema, data)`

Return JSON Patch operations for missing defaults.

`json_schema_update(schema, data)`

Return the document with defaults applied.

`TRY(expression)`

DuckDB wrapper: convert runtime validation errors into NULL.

03 Keep in mind

Make validation policy explicit

Plain validation aborts on nonconforming input. COALESCE(TRY(json_schema_validate(...)), false) gives a filterable flag. Lint your schema first: TRY also hides malformed schemas, so a bad contract could otherwise mark every row invalid.

04 Flag valid and invalid rows

```
SELECT payload, coalesce(try(
  json_schema_validate(
    '{"type":"integer"}', payload
  )), false) AS valid
FROM (VALUES ('5'::JSON), ('"five"'::JSON))
  t(payload);
-- 5: true; "five": false
```

TRY preserves the query when a row violates the schema.

05 Fill a missing property

```
SET VARIABLE schema = '{
  "type":"object",
  "properties":{
    "retries":{"type":"integer","default":3}
  }
}';
SELECT json_schema_validate_schema(
  getvariable('schema'));
SELECT json_schema_update(
  getvariable('schema'), '{}');
-- {"retries":3}
SELECT json_schema_patch(
  getvariable('schema'), '{}');
```

The patch contains an add operation at /retries with value 3.

06 In practice

Draft and defaults

The validator primarily targets draft-07. Test the keywords your contract uses and keep reference targets within the schema. Defaults are applied by patch/update; a successful validate call alone does not rewrite the original JSON.

Full docs & examples ↗ Source ↗

JSONata

Query and reshape JSON with compact expressions

Install & load

```
INSTALL jsonata FROM community;
LOAD jsonata;
```

01 At a glance

Filter nested arrays, construct output objects, and aggregate values within a JSON document. JSONata keeps these transformations in one SQL expression and returns JSON that can be cast, extracted, or expanded by DuckDB.

JSON document → JSONata expression → New JSON shape

02 SQL entry point + expression syntax

`jsonata(expr, data)`

Evaluate an expression against a JSON document.

`jsonata(expr, data, bindings)`

Bind JSON object keys as $variables in the expression.

`Account.Name`

Follow nested fields using dotted paths.

`items[price > $limit]`

Filter array members by a predicate.

`{"total": $sum(prices)}`

Construct an output object with an aggregate.

`[expression]`

Use array construction when a stable array shape matters.

03 Keep in mind

> **Bind values, keep expressions stable**
>
> Pass thresholds and other parameters through the bindings object instead of concatenating them into the expression. Its keys omit the dollar sign: {"limit": 100} is read as $limit inside JSONata.

04 Filter with a bound threshold

```
SELECT jsonata(
  '[items[price > $limit].id]',
  '{"items":[{"id":"A","price":20},' ||
  '{"id":"B","price":120}]}',
  '{"limit":100}'
);
-- ["B"]
```

The outer brackets preserve an array for one matching item.

05 Build a summary object

```
SELECT jsonata(
  '{"customer": name, "total": $sum(prices)}',
  '{"name":"Ada","prices":[12,18,5]}'
);
-- {"customer":"Ada","total":35}
```

Construct several output fields without unpacking the document.

06 Return a SQL number

```
SELECT jsonata('$sum(prices)',
  '{"prices":[12,18,5]}'
)::INTEGER AS total;
-- 35
```

Cast the JSON result before joining or doing typed SQL arithmetic.

07 In practice

> **Pay attention to result shape**
>
> JSONata sequences can collapse to a single value. Wrap a projection in array construction when downstream SQL expects JSON[]. The function returns JSON; cast numbers explicitly or use DuckDB JSON extractors for strings.

Full docs & examples ↗ Source ↗

Lindel

Many dimensions. One sortable integer.

Install & load

```
INSTALL lindel FROM community;
LOAD lindel;
```

01 The four functions

Encode a fixed-size numeric array into an unsigned integer; decode it with the matching curve.

Function	Purpose
`hilbert_encode(a)`	Stronger spatial locality
`morton_encode(a)`	Bit interleaving / Z-order
`hilbert_decode(k,n,f,u)`	Recover Hilbert inputs
`morton_decode(k,n,f,u)`	Recover Morton inputs

```
SELECT
  hilbert_encode([10, 20]::UINTEGER[2]) AS h,
  morton_encode([10, 20]::UINTEGER[2]) AS m;
-- h = 884, m = 408 (both UBIGINT)
```

Cast explicitly: `UINTEGER[2]` is an ARRAY; an uncast `[10, 20]` is a LIST.

02 Choose your input width

Element type	Dimensions
`TINYINT` / `UTINYINT`	1–16
`SMALLINT` / `USMALLINT`	1–8
`INTEGER` / `UINTEGER` / `FLOAT`	1–4
`BIGINT` / `UBIGINT` / `DOUBLE`	1–2

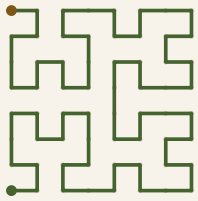
The output uses the smallest unsigned width that fits all component bits:

Input array	Output
`UTINYINT[2]`	`USMALLINT`
`UINTEGER[2]`	`UBIGINT`
`UINTEGER[3]`	`UHUGEINT`
`DOUBLE[2]`	`UHUGEINT`

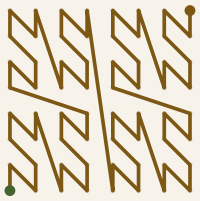
Prepare meaningful coordinates

For numeric locality across signed or floating values, shift and quantize to nonnegative integers. Direct encoding preserves their bit patterns; it does not normalize numeric distance. Keep precision and units consistent.

03 Choose a curve



Hilbert
Fewer long jumps; a good starting point for locality.



Morton / Z-order
Simpler to compute; jumps at quadrant boundaries.

04 Decode & round-trip

```
SELECT hilbert_decode(
  hilbert_encode([10, 20]::UINTEGER[2]),
  2, false, true
) AS point;
-- [10, 20]
```

n = dimensions; `f` = return floats; `u` = return unsigned integers. For signed integers use `false, false`; for floats use `true, false`. Preserve the key's original SQL type. Fixed v1.5 builds support signed 1-D round trips; older 6435106 rejects them.

05 Write a clustered Parquet

```
COPY (
  SELECT x, y
  FROM (VALUES (3, 5), (1, 1), (7, 0),
               (2, 6), (5, 3), (0, 4))
       AS points(x, y)
  ORDER BY
    hilbert_encode([x, y]::UINTEGER[2])
) TO 'points.parquet' (FORMAT PARQUET);
```

Read using ordinary predicates on the original columns:

```
SELECT * FROM 'points.parquet'
WHERE x BETWEEN 1 AND 3
  AND y BETWEEN 1 AND 6;
```

The tiny fixture shows syntax. With many row groups, the ordering can improve min/max pruning; benchmark your data and filters. This is a file layout, not an index.

Full docs & examples ↗ Source ↗

Marisa

Static dictionaries with fast prefix lookup.

Install & load

```
INSTALL marisa FROM community;
LOAD marisa;
```

01 At a glance

Build a compact MARISA trie from a column of strings. Store the trie as a BLOB and use it for exact membership, autocomplete, or prefix-rule matching.

String column → Trie BLOB → Lookup / prefix

02 Core SQL surface

`marisa_trie(text)`

Aggregate a column of strings into one trie BLOB.

`marisa_lookup(trie, text)`

Test exact membership; returns BOOLEAN.

`marisa_predictive(trie, prefix, limit)`

Return stored strings that begin with the supplied prefix.

`marisa_common_prefix(trie, text, limit)`

Return stored strings that are prefixes of the supplied text.

03 Keep in mind

Two directions of prefix search

Predictive search expands a short input into longer dictionary entries. Common-prefix search finds shorter stored rules matching the start of a longer input.

04 Build a dictionary

```
CREATE TABLE dictionary AS
SELECT marisa_trie(name) AS trie
FROM (VALUES ('Alice'), ('Megan'),
             ('Melissa')) AS t(name);
```

One result row stores the entire dictionary.

05 Lookup & autocomplete

```
SELECT marisa_lookup(trie, 'Alice') AS found,
       marisa_predictive(trie, 'Me', 10)
         AS suggestions
FROM dictionary;
```

found is true; suggestions includes Megan and Melissa.

06 Match stored prefixes

```
WITH rules AS (
  SELECT marisa_trie(s) AS trie
  FROM (VALUES ('U'), ('US'), ('USA')) t(s)
)
SELECT marisa_common_prefix(trie, 'USA', 10)
FROM rules;
```

The matching rules are U, US and USA; the limit caps the result count.

07 In practice

Build for a static set

Rebuild the aggregate when the dictionary changes. Normalize case and whitespace before both construction and querying if those differences should be ignored.

Full docs & examples ↗ Source ↗

MiniJinja

Render Jinja-style templates directly from SQL

Install & load

```
INSTALL minijinja FROM community;
LOAD minijinja;
```

01 At a glance

Turn JSON context into rendered text using familiar Jinja variables, filters, loops, and conditions. Generate one HTML fragment, message, or configuration block per row while keeping the source data and rendering query together.

Template → JSON context → Rendered text

02 SQL entry points + template syntax

`minijinja_render(template, ...)`

Render a template with optional named settings.

`minijinja_render_with_context`

Pass template, per-row JSON context, then options.

`autoescape := false`

Disable HTML escaping for plain text and other non-HTML output.

`{{ value | upper }}`

Interpolate a value and apply a filter.

`{% for x in items %}`

Loop through context items; finish with endfor.

`{% if condition %}`

Choose content using if / elif / else / endif.

03 Keep in mind

Choose the output format

HTML autoescape is enabled by default: B&O renders as B&O. Keep it for HTML, or pass autoescape := false for plain text. Escaping for SQL, JSON, and shell syntax is a separate concern.

04 Render a message per row

```
SELECT minijinja_render_with_context(
  'Hello {{ name | upper }}!',
  json_object('name', name)
) AS message
FROM (VALUES ('Ada'), ('Grace')) t(name);
-- Hello ADA! / Hello GRACE!
```

Each row supplies its own context to the same template.

05 Loop through a list

```
SELECT minijinja_render_with_context(
  '{% for x in xs %}{{ x }}' ||
  '{% if not loop.last %}, {% endif %}' ||
  '{% endfor %}',
  '{"xs":["A","B","C"]}'
);
-- A, B, C
```

loop.last makes separators conditional without a trailing comma.

06 Render unescaped plain text

```
SELECT minijinja_render(
  '{{ v }}', '{"v":"B&O"}',
  autoescape := false
);
-- B&O
```

Omit the option for HTML output: B&O.

07 In practice

Keep values in the context

Use minijinja_render_with_context for JSON built from row columns. Keep templates trusted. If generating SQL, bind untrusted data through prepared statements; template interpolation is not a substitute for SQL parameters.

Full docs & examples ↗ Source ↗

QuickJS

Bring JavaScript expressions into DuckDB queries

Install & load

```
INSTALL quickjs FROM community;
LOAD quickjs;
```

01 At a glance

Evaluate a JavaScript expression, call a function with SQL arguments, or expand an array into rows. Use the embedded QuickJS engine for transformations that benefit from JavaScript control flow, string methods, or JSON reshaping.

SQL arguments → JavaScript → Text / JSON / rows

02 Core SQL surface

`SELECT quickjs(code)`

Evaluate JavaScript and return its result as VARCHAR.

`FROM quickjs(code, ...)`

Evaluate an array expression; emit one result row per element.

`quickjs_eval(function, ...)`

Call a JavaScript function and return JSON.

`arg0, arg1, ...`

Access positional arguments in the table form.

`parsed_arg0, ...`

Access a parsed copy of JSON strings in the table form.

`JSON.parse(text)`

Parse a JSON string argument inside quickjs_eval.

03 Keep in mind

Match the return shape

Scalar quickjs returns VARCHAR, quickjs_eval returns JSON, and FROM quickjs returns rows in a result column. Cast or extract the result explicitly when the surrounding SQL needs a numeric or structured value.

04 Call a function with row values

```
SELECT n, quickjs_eval(
  'x ⇒ x * x', n
)::INTEGER AS square
FROM range(1, 4) t(n);
-- 1, 4, 9
```

Use function parameters for per-row SQL arguments.

05 Unpack a transformed array

```
SELECT result::INTEGER AS value
FROM quickjs(
  'parsed_arg0.map(x ⇒ x * arg1)',
  '[1,2,3]', 3
);
-- 3, 6, 9
```

The table form exposes raw and parsed arguments.

06 Reshape a JSON document

```
SELECT quickjs_eval(
  'text ⇒ { const d = JSON.parse(text);' ||
  'return {user: d.name, active: true}; }',
  '{"name":"Ada"}'
);
-- {"user":"Ada","active":true}
```

JSON arrives as text here; parse before accessing its fields.

07 In practice

Keep the interpreter focused

JavaScript runs per row and costs more than native SQL. Filter first and reserve it for transformations that need it. This is an embedded language runtime, without browser or Node APIs; use trusted script text.

Full docs & examples ↗ Source ↗

Radio

Live channel messages meet relational queries.

Install & load

```
INSTALL radio FROM community;
LOAD radio;
```

01 At a glance

Subscribe to WebSocket or Redis Pub/Sub endpoints, query buffered messages, and queue replies from DuckDB. The subscription URL identifies the feed throughout its lifecycle, while background I/O keeps network activity separate from buffer scans.

Live channel → Memory buffer → DuckDB rows

02 Subscription lifecycle

`CALL radio_subscribe(url)`

Open a feed; receive_message_capacity tunes its buffer size.

`radio_subscriptions()`

Inspect active feeds and transmission success or failure counters.

`radio_received_messages()`

Read buffered messages across all subscriptions.

`radio_subscription_received_messages(url)`

Read the buffer for one subscription URL.

`CALL radio_listen(first, wait)`

Wait for arrivals, bounded by an INTERVAL timeout.

`CALL radio_transmit_message(…)`

Queue URL, channel, BLOB payload, attempt count and retry interval.

`CALL radio_unsubscribe(url)`

Close a feed using the exact subscription URL.

03 Keep in mind

A buffer is not durable storage

Subscriptions and messages live in process memory. Capacity limits can evict older messages; process exit discards state. Persist records you need in a DuckDB table and define deduplication around subscription_id and message_id.

04 Subscribe, wait and inspect

REQUIRES SETUP

```
SET VARIABLE feed = 'ws://localhost:8765/';
CALL radio_subscribe(getvariable('feed'));
CALL radio_listen(true, INTERVAL '5 seconds');
SELECT receive_time, channel,
       decode(message) AS text
FROM radio_received_messages()
WHERE subscription_url = getvariable('feed')
ORDER BY receive_time DESC
LIMIT 20;
```

Requires a WebSocket endpoint on localhost:8765 that emits UTF-8 text messages.

05 Queue a reply and inspect delivery

REQUIRES SETUP

```
CALL radio_transmit_message(
  getvariable('feed'), 'main',
  '{"type":"ack"}'::BLOB,
  3, INTERVAL '500 milliseconds'
);
SELECT transmit_successes, transmit_failures
FROM radio_subscriptions();
```

Poll until delivery completes before calling radio_unsubscribe. Your server defines channel semantics.

06 Check the extension build

```
SELECT radio_version();
```

Local diagnostic; no subscription is opened.

07 In practice

Payloads and delivery

message is a BLOB. Use decode(message) for UTF-8 text, then JSON functions if appropriate. Transmit calls queue work; check subscription counters for delivery progress. A queued message does not imply successful delivery.

Full docs & examples ↗ Source ↗

RapidFuzz

Fuzzy string comparisons beside your data.

Install & load

```
INSTALL rapidfuzz FROM community;
LOAD rapidfuzz;
```

01 At a glance

Score candidate pairs using edit-distance and token-based matching. Normalize text first, select a score that matches your data, then rank or threshold the candidates.

Normalize text → Score candidates → Rank matches

02 Core SQL surface

`rapidfuzz_ratio(a, b)`

General string similarity on a 0–100 scale.

`rapidfuzz_partial_ratio(a, b)`

Find a shorter string within a longer string.

`rapidfuzz_token_sort_ratio(a, b)`

Compare after sorting words; tolerates word-order changes.

`rapidfuzz_token_set_ratio(a, b)`

Compare token sets; discounts repeated words.

`rapidfuzz_osa_distance(a, b)`

Edit distance with adjacent transpositions; smaller is closer.

`rapidfuzz_hamming_distance(a, b)`

Count positional differences between strings.

`rapidfuzz_jaro_winkler_normalized_similarity`

Normalized similarity from 0 to 1; larger is closer.

03 Keep in mind

Score scales differ

Ratio functions use 0–100. Normalized similarities use 0–1. Distance functions point the other way: zero means an exact match. A score is not a match probability.

04 Reordered words

```
SELECT rapidfuzz_token_sort_ratio(
  'world hello', 'hello world'
) AS score;
-- 100
```

Token sorting handles word order without changing the score scale.

05 Find the closest candidate

```
SELECT name,
  rapidfuzz_ratio('query farm', lower(name))
    AS score
FROM (VALUES ('Query Farm'), ('Query Firm'),
             ('Quiet Barn')) AS t(name)
ORDER BY score DESC
LIMIT 2;
```

This small fixture is self-contained. Apply the same pattern to blocked pairs.

06 Measure a transposition

```
SELECT rapidfuzz_osa_distance(
  'form', 'from'
) AS edits;
-- 1
```

OSA counts one adjacent transposition as one edit.

07 In practice

Reduce candidate pairs first

A cross join compares every pair. Block on useful fields before scoring large tables. Explicitly lower-case, trim, or standardize punctuation when your matching rules require it.

Full docs & examples ↗ Source ↗

Redis

Read hot data. Analyze it with SQL.

Install & load

```
INSTALL redis FROM community;
LOAD redis;
```

01 At a glance

Query Redis strings, hashes and lists from DuckDB, or snapshot operational data into local tables for analysis. Every Redis call selects a named DuckDB secret, keeping connection details separate from the data-access expression.

Redis keys → SQL functions → Local analysis

02 Useful Redis operations

`redis_get(key, secret)`

Read a string; redis_set writes a key and value.

`redis_hget(key, field, secret)`

Read one hash field; redis_hset writes a field value.

`redis_hgetall(key, secret)`

Return a table with field and value columns.

`redis_lrange_table(key, start, stop, secret)`

Return list positions and values; stop = -1 includes the end.

`redis_scan(cursor, pattern, count, secret)`

Return cursor:keys_csv; repeat until the returned cursor is zero.

`redis_hscan_over_scan(pattern, fields, n, secret)`

Stream key, field and value rows from matching hashes.

`redis_exists(key, secret)`

Check existence; redis_type reports the stored value type.

`redis_expire(key, seconds, secret)`

Set a TTL; redis_expireat takes a Unix timestamp and redis_ttl reads the remaining seconds.

03 Keep in mind

Choose table-shaped results

Use redis_hgetall and redis_lrange_table for relational results. Scalar redis_mget and redis_lrange return comma-separated strings. Missing string or hash fields can return an empty string; check existence when that distinction matters.

04 Name the connection

```
CREATE SECRET quickref_redis (
  TYPE redis,
  host 'localhost',
  port '6379'
);
```

For a local Redis server; add its required password to the secret configuration.

05 Read a hash as rows

REQUIRES SETUP

```
SELECT field, value
FROM redis_hgetall(
  'user:42', 'quickref_redis'
);
```

Requires the server and a Redis hash at user:42.

06 Snapshot matching hashes

REQUIRES SETUP

```
CREATE TABLE users_snapshot AS
SELECT key, field, value
FROM redis_hscan_over_scan(
  'user:*', '*', 100, 'quickref_redis'
);
SELECT key, count(*) AS fields
FROM users_snapshot
GROUP BY key;
```

Use a pattern containing only hashes. Subsequent analysis reads the local snapshot.

07 In practice

Scope scans and connections

Prefer cursor-based SCAN over KEYS on large keyspaces. A scan is not a transactionally consistent snapshot while Redis changes. The documented transport is single-node plain TCP; configure the connection and network boundary accordingly.

Full docs & examples ↗ Source ↗

Shellfs

Read and write shell pipelines through file paths

Install & load

```
INSTALL shellfs FROM community;
LOAD shellfs;
```

01 At a glance

Use a shell command as a DuckDB file source or sink. A trailing pipe reads command output; a leading pipe sends COPY output to a command. Useful for local tools that already produce or consume CSV and JSON.

Local command → Streaming pipe → DuckDB rows

02 Pipe paths + SQL integration

`'command |'`

Read the command's standard output as a file.

`'| command'`

Write bytes to the command's standard input.

`read_csv(pipe_path, ...)`

Parse streamed CSV with normal DuckDB reader options.

`read_json(pipe_path, ...)`

Parse JSON emitted by a command.

`COPY (...) TO pipe_path`

Stream query results into a command; specify the format.

`SET ignore_sigpipe = true`

Optional setting to ignore SIGPIPE; default is false.

03 Keep in mind

Pipe placement sets direction

Place | at the end to read, or at the beginning to write. Commands run through the host shell with the process's permissions. Keep paths trusted and never construct commands by concatenating untrusted input.

04 Query a command's CSV output

```
SELECT sum(n) AS total
FROM read_csv(
  'printf "n\n1\n2\n3\n" |',
  header = true
);
-- 6
```

Runs local printf; the trailing pipe makes it a readable path.

05 Compress an export with gzip

```
COPY (SELECT range AS n FROM range(1, 4))
TO '| gzip > /tmp/qf-shellfs.csv.gz'
(FORMAT CSV, HEADER, COMPRESSION UNCOMPRESSED);
```

Requires gzip and writable /tmp. Disable COPY compression so only gzip compresses the stream.

06 Read the archive back

```
SELECT sum(n) AS total
FROM read_csv('/tmp/qf-shellfs.csv.gz');
-- 6
```

Run after the export above; DuckDB detects gzip from the filename.

07 In practice

Treat the source as a stream

Pipes do not behave like seekable local files. CSV and newline-delimited JSON are natural fits. Set explicit parsing options when needed, and ensure the command and any prerequisite tools exist on the host.

Full docs & examples ↗ Source ↗

Stochastic

Probability, sampling & Monte Carlo. All in SQL.

Install & load

```
INSTALL stochastic FROM community;
LOAD stochastic;
```

01 One naming pattern

```
dist_<family>_<operation>(params..., x)
```

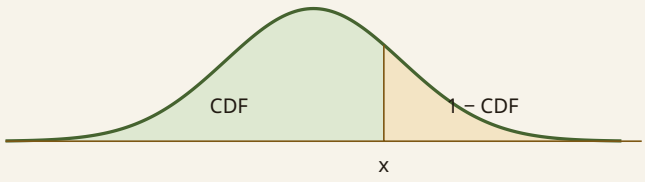
Distribution parameters come first. Add x for density or cumulative probability, p for a quantile, and neither for sampling or properties.

Operation suffix	Returns
`sample`	One random draw
`pdf`	Density / discrete mass[1]
`cdf`	Left tail: P(X ≤ x)
`cdf_complement`	Right tail: P(X > x)
`quantile`	Inverse CDF at p
`quantile_complement`	Inverse right tail at p
`log_pdf`	Natural log of PDF
`log_cdf`	Natural log of CDF
`log_cdf_complement`	Natural log of right tail
`hazard`	PDF / right tail
`chf`	−ln(right tail)

Properties: no trailing x or p

`mean`, `median`, `mode`, `variance`, `stddev`, `skewness`, `kurtosis`, `kurtosis_excess`, `range`, `support`. Availability depends on the family; `range` and `support` return two-element arrays.

02 Read the probability



The CDF is the shaded area left of x. Its complement is the area to the right. For continuous distributions, `pdf(x)` is a density, not P(X = x).

Tail arithmetic

For discrete counts: P(X ≥ k) = `cdf_complement(params..., k-1)`. Call the complement directly rather than subtracting a CDF from 1.

03 Normal: quantiles & CDFs

Parameters are mean and **standard deviation**.

```
SELECT dist_normal_quantile(0, 1, 0.95);
-- 1.6448536269514729
SELECT dist_normal_cdf(0, 1, 1.96);
-- approximately 0.975
```

04 Count probabilities

```
-- P(X ⩾ 5), Poisson(rate = 2.5)
SELECT dist_poisson_cdf_complement(2.5, 4);
-- 0.10882198108584878

-- P(X ⩽ 5), Binomial(n = 10, p = 0.3)
SELECT dist_binomial_cdf(10, 0.3, 5);
-- 0.9526510126000001
```

05 Generate → evaluate → average

Estimate π from uniform points in the square. Materialize draws so every expression reuses the same sampled coordinates.

```
WITH draws AS MATERIALIZED (
  SELECT
    dist_uniform_real_sample(-1, 1) AS x,
    dist_uniform_real_sample(-1, 1) AS y
  FROM range(100000)
)
SELECT 4.0 * AVG(
  (x*x + y*y ⩽ 1)::INTEGER
) AS pi_estimate
FROM draws;
```

Result varies by run and approaches π as the sample size grows.

Keep draws for reproducibility

`_sample` has no seed argument. Save draws with `CREATE TABLE ... AS SELECT ...` and reuse that table for repeatable analysis.

[1] `uniform_int` uses inclusive integer endpoints; see page 2. Discrete quantiles use family-specific rounding.

Full docs & examples ↗ Source ↗

Stochastic

The distribution directory

Use the family token in `dist_<family>_<operation>`. Parameters below are listed in call order; append `x` or `p` when the operation needs it.

06 Continuous families

Family token	Parameters / interpretation
`beta`	alpha, beta Positive shapes; proportions
`cauchy`	location, scale Heavy tails; no mean / variance
`chi_squared`	df Positive degrees of freedom
`exponential`	rate Mean waiting time = 1/rate
`extreme_value`	location, scale Gumbel; block maxima
`fisher_f`	df1, df2 Ratio of scaled variances
`gamma`	shape, scale Both positive; mean = shape × scale
`laplace`	location, scale Symmetric, sharper peak
`logistic`	location, scale Symmetric, heavier tails
`lognormal`	meanlog, sdlog Parameters of ln(X)
`normal`	mean, stddev Location and spread
`pareto`	scale, shape * Lower bound and tail shape
`rayleigh`	scale Nonnegative magnitudes
`students_t`	df Standard t; positive df
`uniform_real`	min, max min < max
`weibull`	shape, scale Both positive; failure times

* Pareto's runtime order is (scale, shape). Fixed v1.5 labels match this order. Older 234c881 labels incorrectly say (shape, minimum).

07 Discrete sampling families

Family token	Parameters / interpretation
`bernoulli`	p A Boolean draw; $0 \le p \le 1$
`binomial`	trials, p Successes in a fixed trial count
`geometric`	p Failures before first success
`negative_binomial`	successes, p Failures before target successes
`poisson`	rate Positive expected event count
`uniform_int`	min, max Integer draws; both ends included

Integer uniform: inclusive endpoints

Fixed v1.5 analytics match integer sampling. For bounds 1 to 6, PMF = 1/6, CDF at 3 = 0.5, mean = 3.5 and variance = 35/12. Older 234c881 uses incorrect continuous formulas and truncates moments.

08 Make a synthetic dataset

```
CREATE TABLE synthetic_users AS
SELECT
  i AS user_id,
  dist_normal_sample(170, 8)
    AS height_cm,
  dist_poisson_sample(3.5) AS sessions,
  dist_bernoulli_sample(0.12)
    AS converted
FROM range(10000) AS t(i);
```

Each row receives fresh draws. Bernoulli returns `BOOLEAN`; other discrete samplers return `BIGINT`; continuous samplers return `DOUBLE`.

09 Check the installed surface

```
SELECT function_name, parameters
FROM duckdb_functions()
WHERE function_name LIKE 'dist_normal_%'
ORDER BY function_name;
```

Use positive scales and standard deviations. Properties may be undefined at some parameter values; invalid inputs can raise SQL errors.

Full docs & examples ↗ Source ↗

Tera

Render templates from inline text or named files

Install & load

```
INSTALL tera FROM community;
LOAD tera;
```

01 At a glance

Render JSON context with Tera's variables, filters, loops, and conditionals. Keep short templates in SQL, or load named files for larger templates and inheritance. Results are ordinary text columns ready for export or further processing.

Text or file → JSON context → Rendered text

02 SQL entry point + template syntax

`tera_render(template)`

Render inline template text without context.

`tera_render(template, context)`

Render with a JSON object supplying variables.

`autoescape := false`

Turn off HTML escaping for non-HTML output.

`template_path := glob`

Load template files; the first argument becomes a name.

`{{ value | upper }}`

Interpolate a value and apply a Tera filter.

`{% for x in items %}`

Loop through context items; finish with endfor.

03 Keep in mind

Choose inline text or a file

Without template_path, the first argument is template source. With a file glob, use a matching template name such as email.html. Files support includes, inheritance, and reusable template authoring outside SQL.

04 Render HTML from context

```
SELECT tera_render(
  '<p>{{ name | upper }}</p>',
  '{"name":"Ada & Grace"}'
);
-- <p>ADA &amp;amp; GRACE</p>
```

HTML escaping applies to the interpolated value.

05 Render a plain-text list

```
SELECT tera_render(
  '{% for x in xs %}{{ x }}' ||
  '{% if not loop.last %}, {% endif %}' ||
  '{% endfor %}',
  '{"xs":["B&O","Acme"]}',
  autoescape := false
);
-- B&O, Acme
```

Use loop.last for separators; disable autoescape for plain text.

06 Use a named template file

REQUIRES SETUP

```
SELECT tera_render(
  'email.html', '{"name":"Ada"}',
  template_path := './templates/*.html'
);
```

Requires ./templates/email.html, for example: Hello {{ name }}!

07 In practice

Escaping follows the output

HTML autoescape is enabled by default. Turn it off for plain text or configuration output, and keep templates trusted. For generated SQL, pass untrusted values through prepared-statement parameters instead of interpolating them.

Full docs & examples ↗ Source ↗

Textplot

Small charts inside your query results.

Install & load

```
INSTALL textplot FROM community;
LOAD textplot;
```

01 At a glance

Render a numeric sequence or progress value as terminal-friendly text. Keep the underlying values for sorting and computation; add plots as a compact visual summary.

Numbers → Plot function → VARCHAR chart

02 Core SQL surface

`tp_bar(value, ...)`

Render one value against a configured range.

`tp_sparkline(values, ...)`

Render a DOUBLE list as a compact sequence.

`tp_density(values, ...)`

Render the distribution of a DOUBLE list.

`tp_qr(text, ...)`

Render text as an ASCII or Unicode QR code.

`width := n`

Control chart width with a named argument.

`array_agg(value ORDER BY time)`

Build an ordered sequence before rendering a sparkline.

03 Keep in mind

Match the terminal

Use ASCII styles where Unicode width or emoji rendering is unreliable. Dense characters can look different across terminal fonts and exported logs.

04 An ASCII progress bar

```
SELECT tp_bar(0.7, width := 15,
  "on" := '#', "off" := '-');
```

Quote reserved named arguments such as on and off.

05 A sequence & a distribution

```
SELECT tp_sparkline([1, 3, 2, 5, 4, 6],
  theme := 'ascii_basic');

SELECT tp_density([1, 5, 3, 8, 2],
  style := 'ascii');
```

Sparklines preserve sequence; density summarizes the distribution.

06 Aggregate in order

```
SELECT tp_sparkline(
  array_agg(value ORDER BY step),
  theme := 'ascii_basic'
) AS trend
FROM (VALUES (1, 4.0), (2, 7.0),
             (3, 5.0)) AS t(step, value);
```

Use ORDER BY inside array_agg so chronology is explicit.

07 In practice

A picture is a string

Plot functions return VARCHAR. Aggregate and order the numeric data before plotting; sort results on the numbers, not on the rendered bars.

Full docs & examples ↗ Source ↗

Tributary

Query Kafka topics from DuckDB.

Install & load

```
INSTALL tributary FROM community;
LOAD tributary;
```

01 At a glance

Read Kafka topic messages as rows with topic, partition, offset and BLOB payload columns. Discover cluster metadata, decode application payloads, and materialize a local table for joins and repeatable downstream analysis.

Kafka topic → Message rows → SQL analysis

02 Reads & client configuration

`tributary_scan_topic(topic, …)`

Read a topic as metadata columns and raw message bytes.

`tributary_metadata(…)`

Inspect broker and topic arrays without scanning a topic.

`tributary_version()`

Identify the loaded extension build.

`"bootstrap.servers"`

Named parameter listing Kafka bootstrap host:port addresses.

`"group.id"`

Choose a dedicated consumer group for the analysis.

`"enable.auto.commit"`

Set to 'false' when analytics must not auto-commit offsets.

`"security.protocol"`

Configure PLAINTEXT, SSL or SASL transport for your broker.

03 Keep in mind

Read the payload deliberately

Quote the offset column as "offset". message is a BLOB; decode(message) interprets UTF-8, and JSON extraction requires valid JSON payloads. Broker client options use quoted librdkafka names, including their dots.

04 Discover topics

REQUIRES SETUP

```
SELECT unnest(topics).name AS topic
FROM tributary_metadata(
  "bootstrap.servers" := 'localhost:9092'
);
```

Requires a compatible Tributary build and a Kafka broker on localhost:9092.

05 Snapshot a partition range

REQUIRES SETUP

```
CREATE TABLE topic_snapshot AS
SELECT partition, "offset", message
FROM tributary_scan_topic('events',
  "bootstrap.servers" := 'localhost:9092',
  "group.id" := 'quick-reference',
  "enable.auto.commit" := 'false'
)
WHERE partition = 0
  AND "offset" BETWEEN 0 AND 99;
SELECT "offset",
       decode(message)::JSON->>'user_id' AS id
FROM topic_snapshot;
```

Assumes events contains UTF-8 JSON messages. Add broker authentication when required.

06 In practice

Plan the read

Use a dedicated consumer group and restrict the topic, partition and offset range for ad-hoc work. This is a consumer interface; publishing belongs in a producer. Community package availability varies by DuckDB version and platform.

Full docs & examples ↗ Source ↗

TSID

Generate sortable IDs and recover their timestamps

Install & load

```
INSTALL tsid FROM community;
LOAD tsid;
```

01 At a glance

Create time-prefixed identifiers as 32-character hexadecimal strings and recover their embedded timestamps. Use them as internal keys, recent-first sort values, or pagination cursors when generation time is useful alongside a compact textual identifier.

Generate ID → Store as VARCHAR → Sort or decode

02 Core SQL patterns

`tsid()`

Generate a new identifier as a 32-character VARCHAR.

`tsid_to_timestamp(id)`

Decode generation time as a timezone-free UTC TIMESTAMP.

`DEFAULT tsid()`

Generate an ID when an INSERT omits the column.

`ORDER BY id DESC`

List IDs in descending time-prefix order.

`WHERE id < cursor`

Continue descending pagination from the last returned ID.

03 Keep in mind

Generation time is not event time

The embedded timestamp records ID generation. Keep a separate event timestamp if events arrive late or describe earlier activity. Concurrent generation and clock changes also mean ID order is not a global transaction-order guarantee.

04 Generate keys automatically

```
CREATE TEMP TABLE events (
  id VARCHAR DEFAULT tsid() PRIMARY KEY,
  action VARCHAR
);
INSERT INTO events(action) VALUES
  ('login'), ('click'), ('logout');
SELECT id, tsid_to_timestamp(id) AS created_utc
FROM events
ORDER BY id;
```

The default generates one ID for each inserted row.

05 Page through recent IDs

```
SELECT * FROM events
ORDER BY id DESC LIMIT 2;

SET VARIABLE cursor = (
  SELECT id FROM events
  ORDER BY id DESC LIMIT 1 OFFSET 1
);
SELECT * FROM events
WHERE id < getvariable('cursor')
ORDER BY id DESC LIMIT 2;
```

Run after the first example; the cursor comes from the previous page's last row.

06 In practice

Treat the format as its own type

Store these IDs as VARCHAR and decode them with tsid_to_timestamp. This extension's format is library-specific; do not assume compatibility with UUID, ULID, or other TSID libraries. Anyone holding an ID can recover its generation time.

Full docs & examples ↗ Source ↗

VGI

Attach a worker. Query its capabilities.

Install & load

```
INSTALL vgi FROM community;
LOAD vgi;
```

01 At a glance

Vector Gateway Interface connects DuckDB to workers written in other languages. A worker advertises catalogs, tables and functions; attach a catalog and query it alongside local data over a subprocess or HTTP connection.

Worker code → Arrow IPC → DuckDB SQL

02 Worker access & inspection

`vgi_catalogs(location)`

Discover catalog names, options and versions before attaching.

`ATTACH catalog (TYPE vgi, LOCATION …)`

Select the advertised catalog and the worker that serves it.

`vgi_table_function(worker, fn, args)`

Call a worker table function directly; args is a LIST.

`vgi_oauth_identity()`

Inspect authenticated identity for attached catalogs.

`vgi_worker_pool()`

List reusable subprocess workers and their process identifiers.

`vgi_worker_pool_stats()`

Inspect worker reuse through per-path hits and misses.

`CALL vgi_clear_cache()`

Clear cached catalog metadata for attached VGI catalogs.

03 Keep in mind

Catalog name and location differ

The ATTACH identifier is a catalog advertised by the worker. LOCATION identifies its URL or executable. Replace example catalog, schema and table names with discovery results. Pushdown and writable tables depend on the worker's capabilities.

04 Discover a hosted worker

REQUIRES SETUP

```
SELECT catalog, source_url
FROM vgi_catalogs(
  'https://worker.example.com/vgi/'
);
```

Requires a compatible VGI build; replace the URL with your worker endpoint.

05 Attach and query

REQUIRES SETUP

```
ATTACH 'analytics' AS app (
  TYPE vgi,
  LOCATION 'https://worker.example.com/vgi/'
);
SHOW TABLES FROM app.main;
SELECT * FROM app.main.events
LIMIT 10;
DETACH app;
```

Assumes the worker advertises analytics with a main.events table.

06 Launch a local worker

MANUAL SETUP

```
ATTACH 'example' AS local_worker (
  TYPE vgi, LOCATION './my_worker',
  pool true
);
SELECT * FROM vgi_worker_pool_stats();
DETACH local_worker;
```

Requires a trusted executable VGI worker advertising the example catalog.

07 In practice

Choose where code runs

Local workers execute with your process privileges; use workers you trust. HTTP workers run in their own environment and may require OAuth. Browser builds use HTTP transport. Community extension availability varies by DuckDB version and platform.

Full docs & examples ↗ Source ↗

WebMacro

Share SQL macros through a trusted URL.

Install & load

```
INSTALL webmacro FROM community;
LOAD webmacro;
```

01 At a glance

Load a DuckDB macro definition from an HTTP or HTTPS endpoint and call it in ordinary SQL. Use reviewed, versioned SQL files to distribute reusable scalar expressions and table queries across notebooks and applications.

SQL at a URL → Load definition → Call the macro

02 Loading & macro lifecycle

`load_macro_from_url(url)`

Fetch and register a macro; return a status string.

`CREATE MACRO name(args) AS expression`

Define a scalar macro in the hosted SQL file.

`CREATE MACRO name(args) AS TABLE …`

Define a table macro called from a FROM clause.

`CREATE OR REPLACE MACRO …`

Let reloading replace a previously registered definition.

`duckdb_functions()`

Inspect registered macros with function_type = 'macro'.

`DROP MACRO name`

Remove a loaded macro with DuckDB's ordinary lifecycle command.

03 Keep in mind

The SQL file is executable code

Load only reviewed SQL from a trusted host. The extension's keyword checks are not a sandbox. Use HTTPS and immutable revision URLs when reproducibility matters; a branch URL can change after review.

04 Prepare a reusable definition

```
CREATE OR REPLACE MACRO normalize_tag(s)
AS lower(trim(s));
SELECT normalize_tag('  Farm  ') AS tag;
-- Result: farm
```

Test locally, then publish the CREATE statement as normalize_tag.sql on your host.

05 Load the hosted SQL

REQUIRES SETUP

```
SELECT load_macro_from_url(
  'https://example.com/normalize_tag.sql'
);
SELECT normalize_tag('  Farm  ') AS tag;
```

Replace the placeholder URL with the trusted SQL file prepared above.

06 Inspect and remove a macro

```
SELECT function_name, macro_definition
FROM duckdb_functions()
WHERE function_type = 'macro'
  AND function_name = 'normalize_tag';
DROP MACRO normalize_tag;
```

After creation or loading, it behaves like an ordinary DuckDB macro.

07 In practice

Load dependencies first

Each call fetches its URL again. There is no dependency resolver: load prerequisite macros before dependents, and load required extensions before use. Plain GET is the interface; private hosts need an accessible or presigned URL.

Full docs & examples ↗ Source ↗